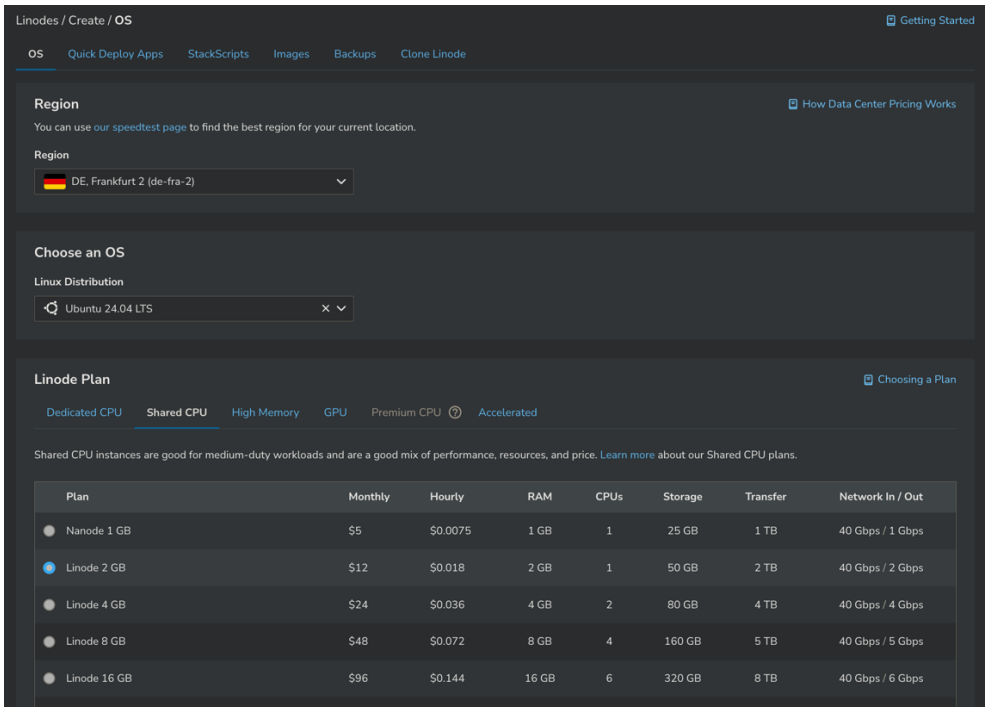
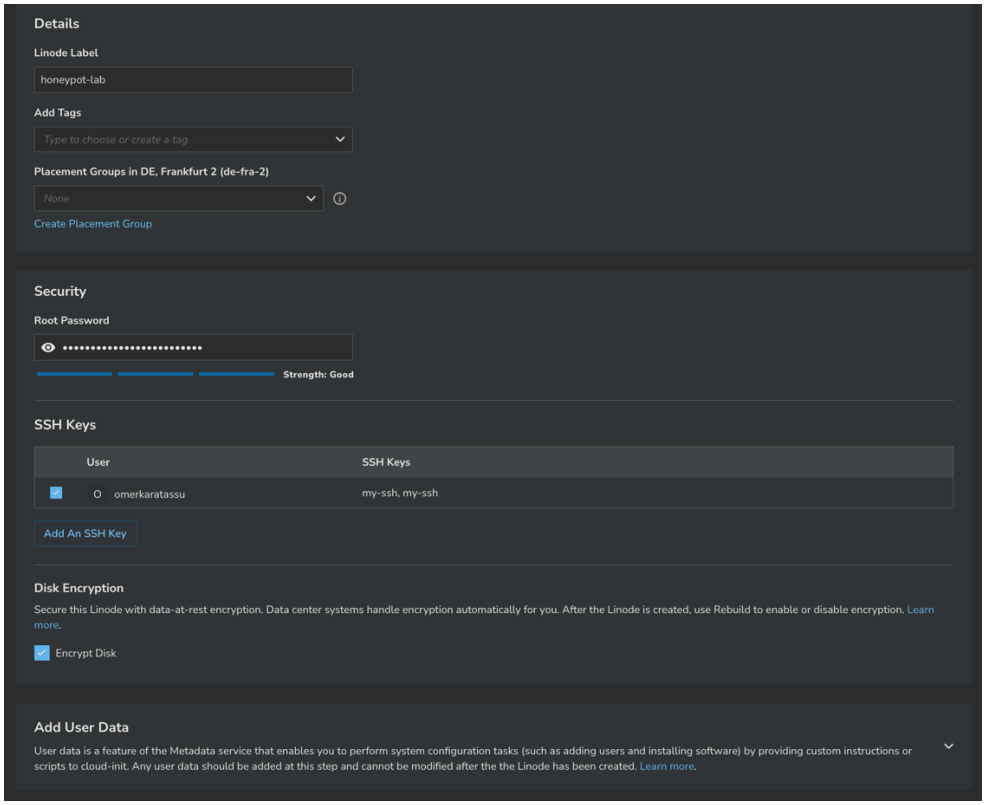
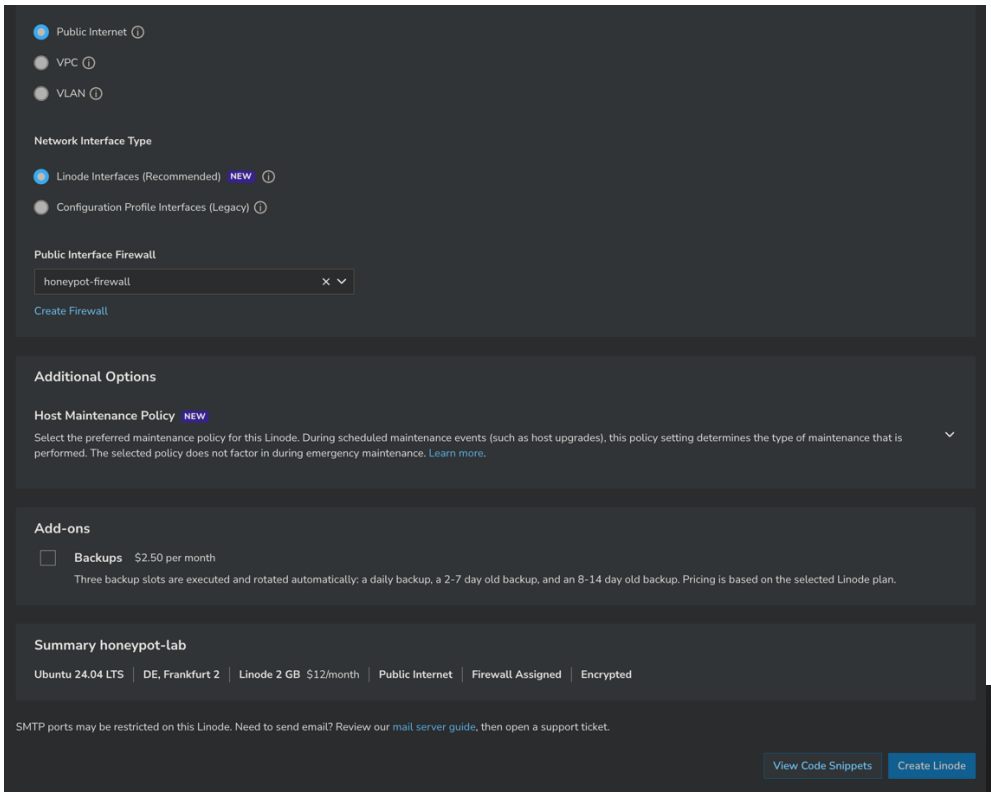
Ömer Karataş - 38160

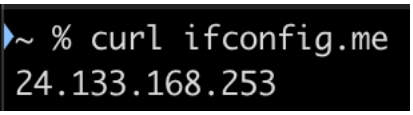
To fulfill the requirements of this lab, I provisioned a remote Virtual Machine (VM) via **Akamai (Linode) Cloud** running **Ubuntu 24.04 LTS** with **2GB of RAM** and **1 vCPU**. This environment was configured with a **Default Outbound Policy** set to **Accept** to allow for necessary updates and repository cloning.

For security, the **Inbound Policy** was set to **Drop** by default, with specific exceptions to facilitate the tasks : **Port 22 (SSH)** was restricted to my local IP (identified via curl ifconfig.me) for secure management, while **Ports 9200 (ElasticPot)**, **9300 (Elastichoney)**, and **9500 (Custom PHP Honeypot)** were opened to all IPv4 traffic to capture and monitor incoming queries as requested in the scenario.









Firewalls / **honeypot-firewall** Docs

Rules | Linodes (1) | NodeBalancers (0)

Inbound Rules

Add An Inbound Rule

Label	Action	Protocol	Port Range	Sources	
ssh-enabled	Accept	TCP	22	24.133.168.253/32	Edit Clone Delete
elasticpod	Accept	TCP	9200	All IPv4, All IPv6	Edit Clone Delete
elastichoney	Accept	TCP	9300	All IPv4, All IPv6	Edit Clone Delete
my-honeypod	Accept	TCP	9500	All IPv4, All IPv6	Edit Clone Delete

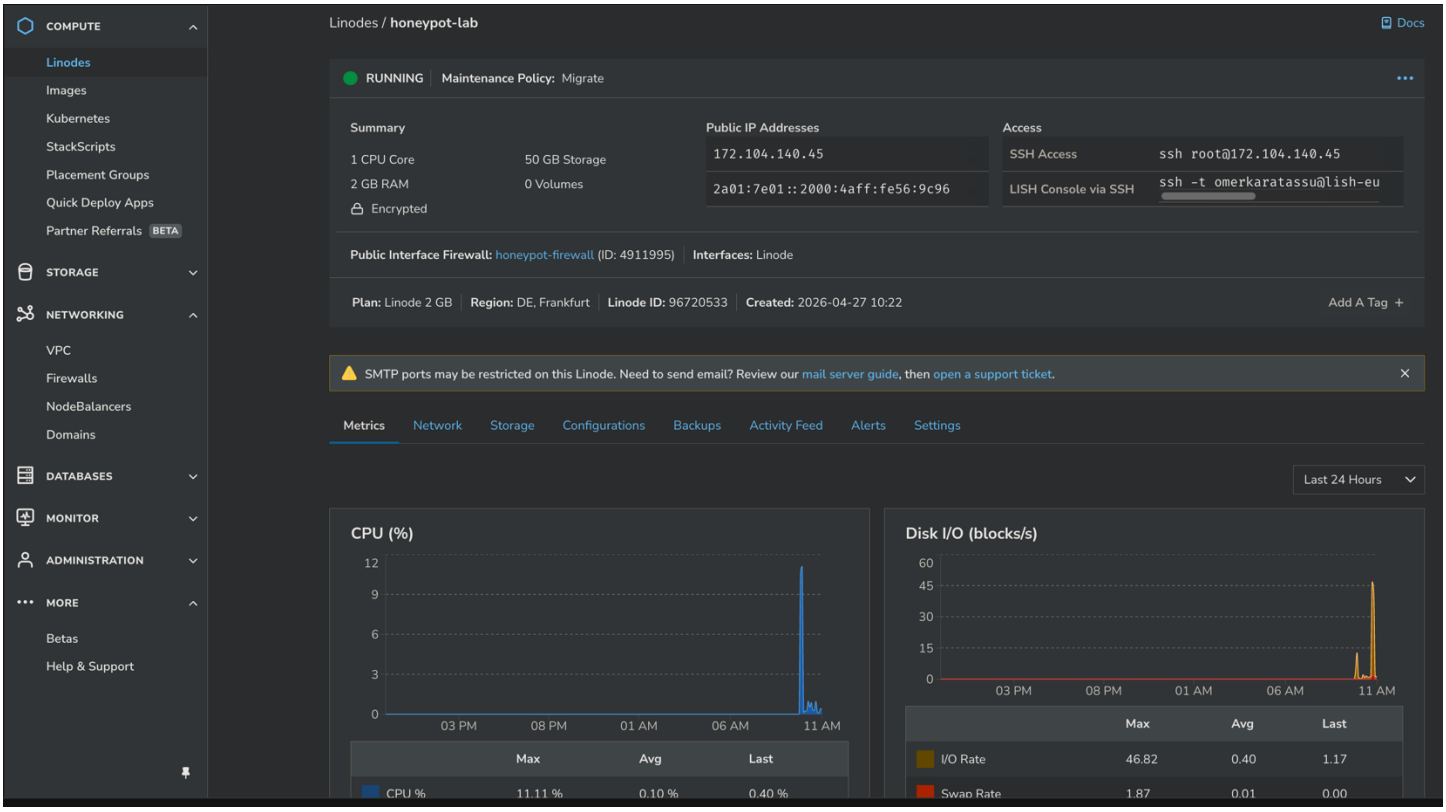
Default inbound policy: This policy applies to any traffic not covered by the inbound rules listed above. Drop

Outbound Rules

Add An Outbound Rule

Label	Action	Protocol	Port Range	Destinations
No outbound rules have been added.				

Default outbound policy: This policy applies to any traffic not covered by the outbound rules listed above. Accept



I successfully accessed the Akamai Cloud VM via SSH using my local terminal. This secure connection to the Ubuntu 24.04.3 LTS instance at 172.104.140.45 establishes the environment for the subsequent technical operations.

```
~ % ssh root@172.104.140.45
The authenticity of host '172.104.140.45 (172.104.140.45)' can't be established.
ED25519 key fingerprint is: SHA256:QZtNaeFtdVxlfMFW2D3PS9YXNiJ1aAqsVJPtSUVoXIc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.104.140.45' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Apr 27 10:23:44 AM UTC 2026

  System load:           0.49
  Usage of /:            4.6% of 48.63GB
  Memory usage:          9%
  Swap usage:            0%
  Processes:             110
  Users logged in:       0
  IPv4 address for eth0: 172.104.140.45
  IPv6 address for eth0: 2a01:7e01::2000:4aff:fe56:9c96

The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

After logging in, I updated the system repositories and upgraded existing packages to their latest versions. I also installed essential development tools and runtimes, including Golang, Python, Git, and Node.js, to prepare the environment for deployment.

```
root@localhost:~# apt update && apt upgrade -y && apt install -y golang python3-pip python3-venv git screen php-cli curl wget python3 nodejs npm htop vim ufw net-tools jq unzip
Get:1 http://mirrors.linode.com/ubuntu noble InRelease
Get:2 http://mirrors.linode.com/ubuntu noble-updates InRelease
Get:3 http://mirrors.linode.com/ubuntu noble-backports InRelease
Get:4 http://mirrors.linode.com/ubuntu noble/main amd64 Packages [1,401 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://mirrors.linode.com/ubuntu noble/main Translation-en [513 kB]
Get:7 http://mirrors.linode.com/ubuntu noble/main amd64 Components [464 kB]
Get:8 http://mirrors.linode.com/ubuntu noble/main amd64 c-n-f Metadata [30.5 kB]
Get:9 http://mirrors.linode.com/ubuntu noble/restricted amd64 Packages [93.9 kB]
Get:10 http://mirrors.linode.com/ubuntu noble/restricted Translation-en [18.7 kB]
Get:11 http://mirrors.linode.com/ubuntu noble/restricted amd64 c-n-f Metadata [416 B]
Get:12 http://mirrors.linode.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:13 http://mirrors.linode.com/ubuntu noble/universe Translation-en [5,982 kB]
Get:14 http://mirrors.linode.com/ubuntu noble/universe amd64 Components [3,871 kB]
Get:15 http://mirrors.linode.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:16 http://mirrors.linode.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:17 http://mirrors.linode.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:18 http://mirrors.linode.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:19 http://mirrors.linode.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8,328 B]
Get:20 http://mirrors.linode.com/ubuntu noble-updates/main amd64 Packages [1,919 kB]
Get:21 http://mirrors.linode.com/ubuntu noble-updates/main Translation-en [345 kB]
Get:22 http://mirrors.linode.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:23 http://mirrors.linode.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.1 kB]
Get:24 http://mirrors.linode.com/ubuntu noble-updates/restricted amd64 Packages [2,981 kB]
Get:25 http://mirrors.linode.com/ubuntu noble-updates/restricted Translation-en [697 kB]
Get:26 http://mirrors.linode.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:27 http://mirrors.linode.com/ubuntu noble-updates/restricted amd64 c-n-f Metadata [556 B]
Get:28 http://mirrors.linode.com/ubuntu noble-updates/universe amd64 Packages [1,685 kB]
Get:29 http://mirrors.linode.com/ubuntu noble-updates/universe Translation-en [324 kB]
Get:30 http://mirrors.linode.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:31 http://mirrors.linode.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.5 kB]
Get:32 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,620 kB]
Get:33 http://mirrors.linode.com/ubuntu noble-updates/multiverse amd64 Packages [32.5 kB]
Get:34 http://mirrors.linode.com/ubuntu noble-updates/multiverse Translation-en [8,180 B]
Get:35 http://mirrors.linode.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:36 http://mirrors.linode.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [512 B]
```

Task 1:

I began the deployment of the first honeypot, Elastichoney, by cloning the official repository from GitHub into the /home directory. After navigating into the project folder, I listed the files to verify the source code and configuration files were present.

To improve the honeypot's effectiveness and make it appear more like a legitimate production server, I modified the config.json file. I updated the instance_name from "Green Goblin" to a more realistic identifier and changed the anonymous setting from false to true to allow unauthenticated access, which is essential for capturing typical Elasticsearch attacks.

```
root@localhost:/# cd /home && git clone https://github.com/jordan-wright/elastichoney.git && cd elastichoney
Cloning into 'elastichoney'...
remote: Enumerating objects: 38, done.
remote: Total 38 (delta 0), reused 0 (delta 0), pack-reused 38 (from 1)
Receiving objects: 100% (38/38), 10.22 KiB | 3.41 MiB/s, done.
Resolving deltas: 100% (17/17), done.
root@localhost:/home/elastichoney# ls
config.json  docker-compose.yml  Dockerfile  LICENSE  main.go  README.md
root@localhost:/home/elastichoney# vi config.json
```

```
root@localhost:/home/elastichoney# cat config.json
{
    "logfile" : "./elastichoney.log",
    "use_remote" : false,
    "remote" : {
        "url" : "http://example.com",
        "use_auth" : false,
        "auth" : {
            "username" : "",
            "password" : ""
        }
    },
    "hpfeeds": {
        "enabled": false,
        "host": "127.0.0.1",
        "port": 10000,
        "ident": "elastichoney",
        "secret": "elastichoney",
        "channel": "elastichoney.events"
    },
    "instance_name" : "ES-PROD-CLUSTER-01",
    "anonymous" : true,
    "spoofed_version"  : "1.4.1",
    "public_ip_url": "http://queryip.net/ip/"
}
```

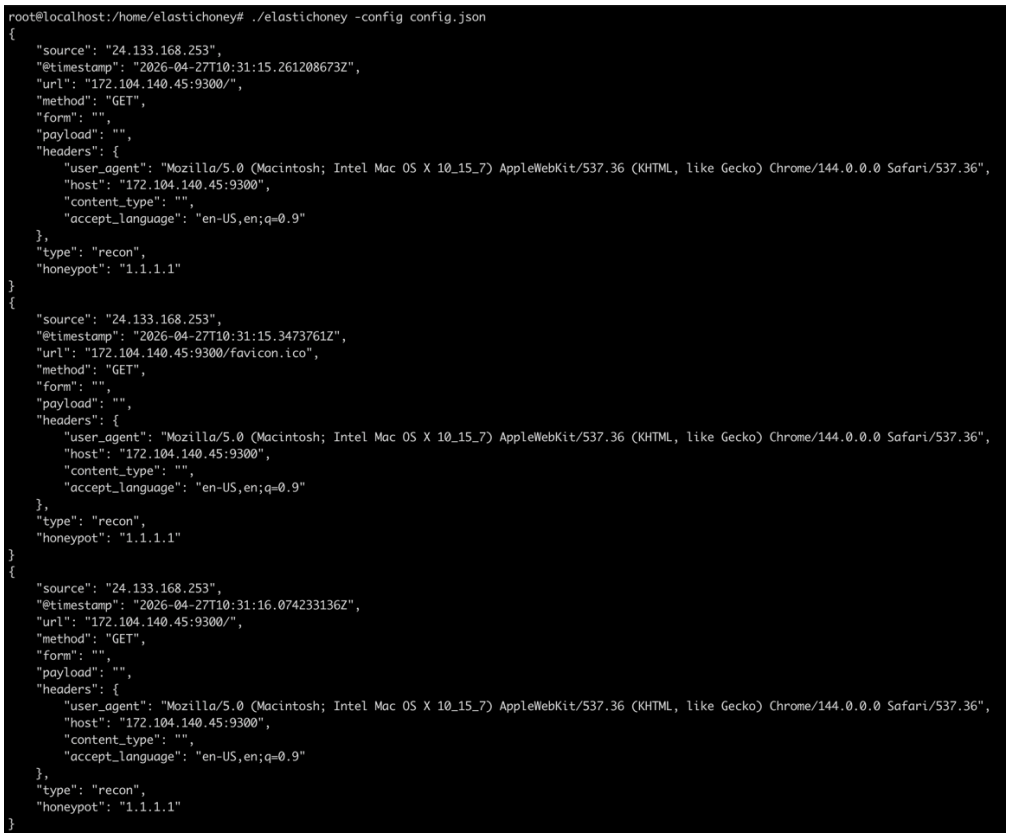
To comply with the requirement of deploying Elastichoney on port 9300, I modified the source code directly. I first used grep to identify all instances of the default port 9200 within the main.go file. Then, I employed the sed command to perform a global search-and-replace, effectively switching the listening port, bound addresses, and logging messages to 9300. Finally, I verified the changes with another grep command to ensure the honeypot was correctly configured to operate on the assigned port.

```
root@localhost:/home/elastichoney# grep -n ":9200" main.go
153:                "http_address" : "inet[/%s:9200]",
185:                  "bound_address" : "inet[/0:0:0:0:0:0:0:0:9200]",
186:                  "publish_address" : "inet[/%s:9200]",
380:              logger.Printf("Listening on :9200")
383:      http.ListenAndServe(":9200", nil)
root@localhost:/home/elastichoney# sed -i 's/:9200/:9300/g' main.go
root@localhost:/home/elastichoney# grep -n ":9200" main.go
root@localhost:/home/elastichoney# grep -n ":9300" main.go
148:                      %s:9300]",
153:                "http_address" : "inet[/%s:9300]",
181:                  "bound_address" : "inet[/0:0:0:0:0:0:0:0:9300]",
182:                  "publish_address" : "inet[/%s:9300]"
185:                  "bound_address" : "inet[/0:0:0:0:0:0:0:0:9300]",
186:                  "publish_address" : "inet[/%s:9300]",
380:              logger.Printf("Listening on :9300")
383:      http.ListenAndServe(":9300", nil)
```

To prepare the application for execution, I initialized the Go module and managed dependencies using go mod init and go mod tidy. This process resolved the necessary external packages, such as the hpfeeds library. Once the environment was ready, I compiled the source code into a standalone executable binary by running the go build command. As seen in the directory listing, the elastichoney binary was successfully generated with the appropriate execution permissions.

```
root@localhost:/home/elastichoney# go mod init elastichoney && go mod tidy && go build
go: creating new go.mod: module elastichoney
go: to add module requirements and sums:
        go mod tidy
go: finding module for package github.com/fw42/go-hpfeeds
go: downloading github.com/fw42/go-hpfeeds v0.0.0-20130602172243-651ce0355974
go: found github.com/fw42/go-hpfeeds in github.com/fw42/go-hpfeeds v0.0.0-20130602172243-651ce0355974
root@localhost:/home/elastichoney# ls -l
total 8052
-rw-r--r-- 1 root root     602 Apr 27 07:53 config.json
-rw-r--r-- 1 root root     200 Apr 27 07:52 docker-compose.yml
-rw-r--r-- 1 root root      56 Apr 27 07:52 Dockerfile
-rwxr-xr-x 1 root root 8198586 Apr 27 08:00 elastichoney
-rw-r--r-- 1 root root     102 Apr 27 08:00 go.mod
-rw-r--r-- 1 root root     227 Apr 27 08:00 go.sum
-rw-r--r-- 1 root root    1081 Apr 27 07:52 LICENSE
-rw-r--r-- 1 root root   12639 Apr 27 07:56 main.go
-rw-r--r-- 1 root root    2091 Apr 27 07:52 README.md
```

I launched the honeypot using the command ./elastichoney -config config.json, which immediately began monitoring for incoming requests on port 9300. To verify the deployment, I accessed the server's IP address (172.104.140.45) via a web browser, confirming that the honeypot successfully served the spoofed Elasticsearch node information. The terminal logs captured these interactions in real-time, displaying detailed JSON metadata for each request, including the source IP, user agent, and specific endpoints accessed such as /_nodes and favicon.ico.



```
{
    "source": "24.133.168.253",
    "@timestamp": "2026-04-27T10:31:16.143640261Z",
    "url": "172.104.140.45:9300/favicon.ico",
    "method": "GET",
    "form": "",
    "payload": "",
    "headers": {
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36",
        "host": "172.104.140.45:9300",
        "content_type": "",
        "accept_language": "en-US,en;q=0.9"
    },
    "type": "recon",
    "honeypot": "1.1.1.1"
}
{
    "source": "24.133.168.253",
    "@timestamp": "2026-04-27T10:31:38.271584858Z",
    "url": "172.104.140.45:9300/_nodes",
    "method": "GET",
    "form": "",
    "payload": "",
    "headers": {
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36",
        "host": "172.104.140.45:9300",
        "content_type": "",
        "accept_language": "en-US,en;q=0.9"
    },
    "type": "recon",
    "honeypot": "1.1.1.1"
}
{
    "source": "24.133.168.253",
    "@timestamp": "2026-04-27T10:31:38.356064272Z",
    "url": "172.104.140.45:9300/favicon.ico",
    "method": "GET",
    "form": "",
    "payload": "",
    "headers": {
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36",
        "host": "172.104.140.45:9300",
        "content_type": "",
        "accept_language": "en-US,en;q=0.9"
    },
    "type": "recon",
    "honeypot": "1.1.1.1"
}
```


Not Secure 172.104.140.45:9300/_nodes

Killercoda cncf Stack cyber sec udemy-exams pi-d


```
{
"cluster_name" : "elasticsearch",
"nodes" : {
    "x1JG6g9PRHy6ClCOO2-C4g" : {
      "name" : "ES-PROD-CLUSTER-01",
      "transport_address" : "inet[/
                1.1.1.1:9300]",
      "host" : "elk",
      "ip" : "127.0.1.1",
      "version" : "1.4.1",
      "build" : "89d3241",
      "http_address" : "inet[/1.1.1.1:9300]",
      "os" : {
        "refresh_interval_in_millis" : 1000,
        "available_processors" : 12,
        "cpu" : {
          "total_cores" : 24,
          "total_sockets" : 48,
          "cores_per_socket" : 2
        }
      },
      "process" : {
        "refresh_interval_in_millis" : 1000,
        "id" : 2039,
        "max_file_descriptors" : 65535,
        "mlockall" : false
      },
      "jvm" : {
        "version" : "1.7.0_65"
      },
      "network" : {
        "refresh_interval_in_millis" : 5000,
        "primary_interface" : {
          "address" : "1.1.1.1",
          "name" : "eth0",
          "mac_address" : "08:01:c7:3F:15:DD"
        }
      },
      "transport" : {
        "bound_address" : "inet[/0:0:0:0:0:0:0:0:9300]",
        "publish_address" : "inet[/1.1.1.1:9300]"
      },
      "http" : {
        "bound_address" : "inet[/0:0:0:0:0:0:0:0:9300]",
        "publish_address" : "inet[/1.1.1.1:9300]",
        "max_content_length_in_bytes" : 104857600
      }}
    }
}
```


Not Secure 172.104.140.45:9300

odeKloud Killercoda cncf Stack cyber sec udemy-exams pi-devices


```
{
        "status" : 200,
        "name" : "ES-PROD-CLUSTER-01",
        "cluster_name" : "elasticsearch",
        "version" : {
            "number" : "1.4.1",
            "build_hash" : "89d3241d670db65f994242c8e838b169779e2d4",
            "build_snapshot" : false,
            "lucene_version" : "4.10.2"
        },
        "tagline" : "You Know, for Search"
    }
```

To confirm that the honeypot was effectively recording activity, I examined the elastichoney.log file using the cat command. The log file successfully captured multiple interaction entries, documenting the source IP, precise timestamps, and the specific request headers for each connection attempt. Due to the length of the output, the preceding screenshots represent only a portion of the complete execution log.

```
root@localhost:/home/elastichoney# ls
config.json  docker-compose.yml  Dockerfile  elastichoney  elastichoney.log  go.mod  go.sum  LICENSE  main.go  README.md
root@localhost:/home/elastichoney# cat elastichoney.log
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:15.261208673Z","url":"172.104.140.45:9300/","method":"GET","form":"","payload":"","headers"
:{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","host":"172.10
4.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:15.3473761Z","url":"172.104.140.45:9300/favicon.ico","method":"GET","form":"","payload":"",
"headers":{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","host
":"172.104.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:16.074233136Z","url":"172.104.140.45:9300/","method":"GET","form":"","payload":"","headers"
:{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","host":"172.10
4.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:16.143640261Z","url":"172.104.140.45:9300/favicon.ico","method":"GET","form":"","payload":"
","headers":{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","ho
st":"172.104.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:38.271584858Z","url":"172.104.140.45:9300/_nodes","method":"GET","form":"","payload":"","he
aders":{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","host":"
172.104.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
{"source":"24.133.168.253","@timestamp":"2026-04-27T10:31:38.356064272Z","url":"172.104.140.45:9300/favicon.ico","method":"GET","form":"","payload":"
","headers":{"user_agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36","ho
st":"172.104.140.45:9300","content_type":"","accept_language":"en-US,en;q=0.9"},"type":"recon","honeypot":"1.1.1.1"}
```

Task 2:

To begin the installation of the second honeypot, ElasticPot, I installed the necessary system dependencies and Python development libraries. I executed a command to install packages such as python3-dev, libmysqlclient-dev, and build-essential to ensure a compatible environment for the honeypot's requirements. Following the system-level setup, I used pip to install the specific Python dependencies listed in the requirements.txt file, completing the initial configuration phase for the ElasticPot project.

```
root@localhost:/home# apt install -y python3-dev libmysqlclient-dev build-essential pkg-config && pip install -r requirements.txt
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-dev is already the newest version (3.12.3-0ubuntu2.1).
python3-dev set to manually installed.
build-essential is already the newest version (12.10ubuntu1).
build-essential set to manually installed.
pkg-config is already the newest version (1.8.1-2build1).
pkg-config set to manually installed.
The following NEW packages will be installed:
  libmysqlclient-dev libmysqlclient21 libzstd-dev mysql-common
0 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 3,216 kB of archives.
After this operation, 17.3 MB of additional disk space will be used.
Get:1 http://mirrors.linode.com/ubuntu noble/main amd64 mysql-common all 5.8+1.1.0build1 [6,746 B]
Get:2 http://mirrors.linode.com/ubuntu noble-updates/main amd64 libmysqlclient21 amd64 8.0.45-0ubuntu0.24.04.1 [1,255 kB]
Get:3 http://mirrors.linode.com/ubuntu noble-updates/main amd64 libzstd-dev amd64 1.5.5+dfsg2-2build1.1 [364 kB]
Get:4 http://mirrors.linode.com/ubuntu noble-updates/main amd64 libmysqlclient-dev amd64 8.0.45-0ubuntu0.24.04.1 [1,591 kB]
Fetched 3,216 kB in 2s (2,075 kB/s)
Selecting previously unselected package mysql-common.
(Reading database ... 176180 files and directories currently installed.)
Preparing to unpack .../mysql-common_5.8+1.1.0build1_all.deb ...
Unpacking mysql-common (5.8+1.1.0build1) ...
Selecting previously unselected package libmysqlclient21:amd64.
Preparing to unpack .../libmysqlclient21_8.0.45-0ubuntu0.24.04.1_amd64.deb ...
Unpacking libmysqlclient21:amd64 (8.0.45-0ubuntu0.24.04.1) ...
Selecting previously unselected package libzstd-dev:amd64.
Preparing to unpack .../libzstd-dev_1.5.5+dfsg2-2build1.1_amd64.deb ...
```

I cloned the ElasticPot repository from GitHub and navigated into the project directory to begin the setup. After listing the directory contents, I accessed the etc/ folder to manage the configuration files. I then renamed the template file honeypot.cfg.base to honeypot.cfg using the mv command to create the active configuration file required for the honeypot to run.

```
root@localhost:/home# git clone https://github.com/bontchev/elasticpot.git && cd elasticpot
Cloning into 'elasticpot'...
remote: Enumerating objects: 440, done.
remote: Counting objects: 100% (440/440), done.
remote: Compressing objects: 100% (166/166), done.
remote: Total 440 (delta 266), reused 440 (delta 266), pack-reused 0 (from 0)
Receiving objects: 100% (440/440), 87.83 KiB | 9.76 MiB/s, done.
Resolving deltas: 100% (266/266), done.
root@localhost:/home/elasticpot# ls
bin  CHANGELOG.md  core  data  Dockerfile  docs  elasticpot.py  etc  LICENSE  log  output_plugins  README.md  requirements.txt  responses
root@localhost:/home/elasticpot# ls etc/
honeypot.cfg.base  honeypot-launch.cfg.base
root@localhost:/home/elasticpot# cd etc/
root@localhost:/home/elasticpot/etc# mv honeypot.cfg.base honeypot.cfg
root@localhost:/home/elasticpot/etc# ls
honeypot.cfg  honeypot-launch.cfg.base
```

I accessed the honeypot.cfg file using the vi editor to finalize the logging parameters. I manually uncommented the [output_textlog] section by removing the # symbols and updated the enabled status from false to true. These modifications ensure that the honeypot actively writes all audit log entries to the designated text file, enabling persistent data collection for further analysis.

```
root@localhost:/home/elasticpot/etc# vi honeypot.cfg
```

```
# Text output
# This writes audit log entries to a text file
#
#[output_textlog]
#enabled = false
#logfile = log/elasticpot.txt
```

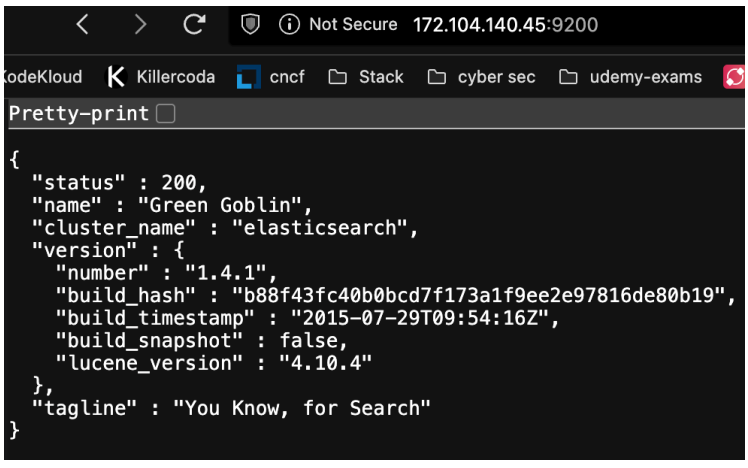
```
# Text output
# This writes audit log entries to a text file
#
[output_textlog]
enabled = true
logfile = log/elasticpot.txt
```

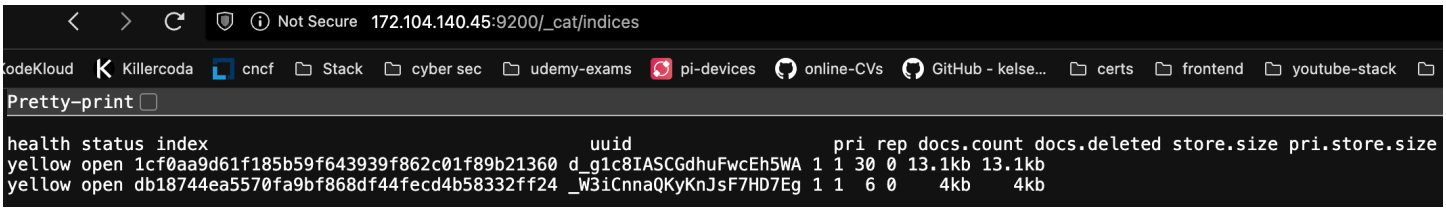
To maintain a clean development environment, I created a Python virtual environment and activated it before installing the project dependencies. Using the pip install -r requirements.txt command, I successfully installed all required libraries, including Twisted, elasticsearch, and mysqlclient. This ensures that all necessary packages are isolated within the virtual environment, preventing potential version conflicts with system-wide Python modules

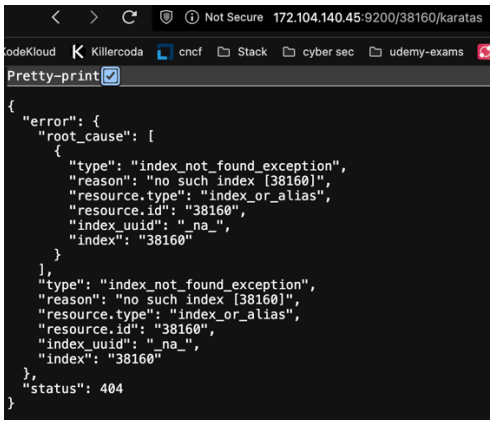
```
root@localhost:/home/elasticpot# python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
Collecting configparser>=3.5.0 (from -r requirements.txt (line 1))
  Downloading configparser-7.2.0-py3-none-any.whl.metadata (5.5 kB)
Collecting service_identity>=18.1.0 (from -r requirements.txt (line 2))
  Downloading service_identity-24.2.0-py3-none-any.whl.metadata (5.1 kB)
Collecting setuptools<45.0.0 (from -r requirements.txt (line 3))
  Downloading setuptools-44.1.1-py2.py3-none-any.whl.metadata (3.7 kB)
Collecting Twisted>=18.9.0 (from -r requirements.txt (line 4))
  Downloading twisted-25.5.0-py3-none-any.whl.metadata (22 kB)
Collecting geoip2>=2.7.0 (from -r requirements.txt (line 5))
  Downloading geoip2-5.2.0-py3-none-any.whl.metadata (19 kB)
Collecting maxminddb>=1.3.0 (from -r requirements.txt (line 6))
  Downloading maxminddb-3.1.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadat
(5.3 kB)
Collecting couchdb (from -r requirements.txt (line 11))
  Downloading CouchDB-1.2-py2.py3-none-any.whl.metadata (930 bytes)
Collecting elasticsearch>=7.7.1 (from -r requirements.txt (line 14))
  Downloading elasticsearch-9.3.0-py3-none-any.whl.metadata (9.0 kB)
Collecting hpfeeds>=3.0.0 (from -r requirements.txt (line 17))
  Downloading hpfeeds-3.1.0-py2.py3-none-any.whl.metadata (1.1 kB)
Collecting influxdb (from -r requirements.txt (line 20))
  Downloading influxdb-5.3.2-py2.py3-none-any.whl.metadata (6.9 kB)
Collecting pymongo (from -r requirements.txt (line 26))
```

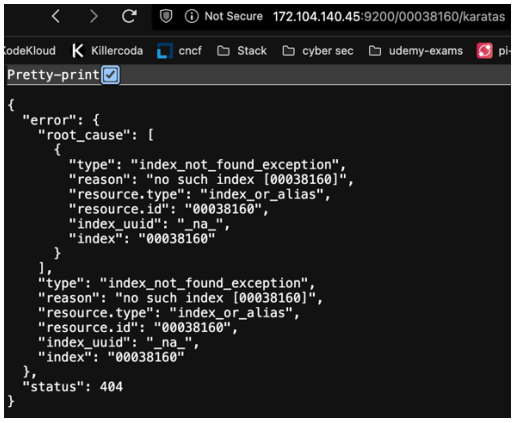
I executed the ElasticPot honeypot on port 9200 and performed the mandatory testing queries from my local machine at IP address 24.133.168.253. To simulate an attack, I first ran the /user/_search?size=1000 query to list available data, followed by identification queries using my student ID and surname (/38160/karatas and /00038160/karatas). As expected, these specific requests returned a 404 error with an "index_not_found_exception," mimicking the behavior of a standard Elasticsearch server.I also verified the decoy's credibility by accessing the /_cat/indices endpoint, which displayed realistic mock indices with "yellow open" status. The honeypot successfully captured all interactions in its logs, including source IPs, timestamps, and request headers. This demonstrates the system's effectiveness in monitoring reconnaissance activities while maintaining high stealth through realistic decoy responses.

```
(venv) root@localhost:/home/elasticpot# python3 elasticpot.py
[2026-04-27 10:38:35.584523Z] Log opened.
[2026-04-27 10:38:35.584782Z] Elasticsearch Honeypot by Vesselin Bontchev
[2026-04-27 10:38:35.584841Z] Loading the plugins...
[2026-04-27 10:38:35.586940Z] Loaded output engine: textlog
[2026-04-27 10:38:35.587151Z] Listening on port 9200.
[2026-04-27 10:38:35.587497Z] Site starting on 9200
[2026-04-27 10:38:35.587671Z] Starting factory <twisted.web.server.Site object at 0x7234f67f0560>
[2026-04-27 10:38:56.636783Z] [INFO] (24.133.168.253:45716): GET: /
[2026-04-27 10:38:56.721737Z] [INFO] (24.133.168.253:45718): GET: /favicon.ico
^[[2026-04-27 10:40:56.740820Z] [INFO] (24.133.168.253:45914): GET: /_cat/indices?v
[2026-04-27 10:40:56.825899Z] [INFO] (24.133.168.253:45915): GET: /favicon.ico
[2026-04-27 10:41:15.335702Z] [INFO] (24.133.168.253:45917): GET: /user/_search?pretty
[2026-04-27 10:41:15.419178Z] [INFO] (24.133.168.253:45916): GET: /favicon.ico
[2026-04-27 10:41:31.626007Z] [INFO] (24.133.168.253:45930): GET: /00038160/karatas?pretty
[2026-04-27 10:41:31.712637Z] [INFO] (24.133.168.253:45931): GET: /favicon.ico
[2026-04-27 10:41:43.016559Z] [INFO] (24.133.168.253:45943): GET: /00038160
[2026-04-27 10:41:43.097801Z] [INFO] (24.133.168.253:45944): GET: /favicon.ico
[2026-04-27 10:41:53.001287Z] [INFO] (24.133.168.253:45952): GET: /38160/karatas?pretty
[2026-04-27 10:41:53.077883Z] [INFO] (24.133.168.253:45953): GET: /favicon.ico
[2026-04-27 10:42:48.845805Z] [INFO] (24.133.168.253:46017): GET: /user/_search?size=1000&pretty
[2026-04-27 10:42:48.933116Z] [INFO] (24.133.168.253:46018): GET: /favicon.ico
```









I confirmed that all test queries were successfully recorded in the persistent audit log. This logging was enabled through the previously mentioned configuration changes to the honeypot.cfg file. By executing cat log/elasticpot.txt, I verified that the file captured the complete sequence of my activities, including the data listing attempt, student identification queries, and index verification. Each entry provides the necessary forensic data, such as the timestamp, my source IP address, and the specific GET requests performed.

```
(venv) root@localhost:/home/elasticpot# ls
bin  CHANGELOG.md  core  data  Dockerfile  docs  elasticpot.py  etc  LICENSE  log  output_plugins  README.md  requirements.txt  responses  venv
(venv) root@localhost:/home/elasticpot# cd log
(venv) root@localhost:/home/elasticpot/log# ls
elasticpot.txt
(venv) root@localhost:/home/elasticpot/log# cat elasticpot.txt
[2026-04-27T10:38:56.636867Z] [localhost] GET / from 24.133.168.253:9200 (Scan)
[2026-04-27T10:38:56.721831Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:40:56.741325Z] [localhost] GET /_cat/indices?v from 24.133.168.253:9200 (Scan)
[2026-04-27T10:40:56.826241Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:15.335797Z] [localhost] GET /user/_search?pretty from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:15.419281Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:31.626118Z] [localhost] GET /00038160/karatas?pretty from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:31.712891Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:43.016750Z] [localhost] GET /00038160 from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:43.098091Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:53.001438Z] [localhost] GET /38160/karatas?pretty from 24.133.168.253:9200 (Scan)
[2026-04-27T10:41:53.077991Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
[2026-04-27T10:42:48.845941Z] [localhost] GET /user/_search?size=1000&pretty from 24.133.168.253:9200 (Scan)
[2026-04-27T10:42:48.933329Z] [localhost] GET /favicon.ico from 24.133.168.253:9200 (Scan)
```

Task 3:

The first unique feature is a Behavioral Detection system that monitors for Remote Code Execution (RCE) and script injection attempts. By using string search functions to scan incoming GET and POST request bodies for critical keywords like "script," "eval," and "painless," the system can identify high-severity attacks in real-time. This is important because it allows the honeypot to capture and log attacker intent beyond simple reconnaissance, providing more meaningful data for security analysis.

The second enhancement involves Advanced Header Mimicry, where the script is configured to return authentic HTTP headers such as "Server: Apache" and "X-elastic-product: Elasticsearch". This ensures that any fingerprinting attempt by an attacker or an automated scanning tool will receive responses consistent with a legitimate Elasticsearch service running behind a standard web server. By matching the expected signatures of a production environment, this feature significantly enhances the decoy's stealth.

Finally, I implemented a Hierarchical Logging System that separates the captured data into three distinct files: a standard log file, an alert-specific file for identified attacks, and a detailed forensic log. This

improves the usability of the honeypot by allowing an analyst to quickly prioritize critical security alerts while still maintaining a comprehensive record of background activity. This structured logging approach ensures that high-value information about attacker behavior is easily accessible without the need to filter through routine scanning logs.

```
root@localhost:/home# mkdir honeyphp
root@localhost:/home# cd honeyphp/
root@localhost:/home/honeyphp# cat > index.php << 'HONEYPOT_EOF'
<?php
/**
 * Enhanced Elasticsearch Honeypot v2.0
 * honeyphp - Port 9500
 * Combined features from Elastichoney and ElasticPot
 */

$CONFIG = [
    'port' => 9500,
    'version' => '7.17.0',
    'cluster_name' => 'elasticsearch-prod',
    'node_name' => 'node-1',
    'node_id' => 'dXpvUEsyUXF6TjRSODEyN2Z2MFg=',
    'log_file' => '/var/log/honeyphp/honeyphp.log',
    'alert_file' => '/var/log/honeyphp/honeyphp_alerts.log',
    'detailed_log' => '/var/log/honeyphp/honeyphp_detailed.log'
];

@mkdir(dirname($CONFIG['log_file']), 0777, true);

function log_attack($type, $details) {
    global $CONFIG;
    $timestamp = date('Y-m-d H:i:s');
    $client_ip = get_client_ip();
    $log_entry = "[{$timestamp}] [{$type}] IP: {$client_ip} | Details: " . json_encode($details) . "\n";
    file_put_contents($CONFIG['log_file'], $log_entry, FILE_APPEND);
}

function get_client_ip() {
    if (!empty($_SERVER['HTTP_CLIENT_IP'])) {
        $ip = $_SERVER['HTTP_CLIENT_IP'];
    } elseif (!empty($_SERVER['HTTP_X_FORWARDED_FOR'])) {
        $ip = explode(',', $_SERVER['HTTP_X_FORWARDED_FOR'])[0];
    } else {
        $ip = $_SERVER['REMOTE_ADDR'];
    }
    return $ip;
}

header('Server: Apache');
header("X-elastic-product: Elasticsearch");

$request_method = $_SERVER['REQUEST_METHOD'];
$request_uri = $_SERVER['REQUEST_URI'];
$request_path = parse_url($request_uri, PHP_URL_PATH);
```

```
root@localhost:/home/honeyphp# ls
index.php
root@localhost:/home/honeyphp# cat index.php
<?php
/**
 * Enhanced Elasticsearch Honeypot v2.0
 * honeyphp - Port 9500
 * Combined features from Elastichoney and ElasticPot
 */

$CONFIG = [
    'port' => 9500,
    'version' => '7.17.0',
    'cluster_name' => 'elasticsearch-prod',
    'node_name' => 'node-1',
    'node_id' => 'dXpvUEsyUXF6TjRSODEyN2Z2MFg=',
    'log_file' => '/var/log/honeyphp/honeyphp.log',
    'alert_file' => '/var/log/honeyphp/honeyphp_alerts.log',
    'detailed_log' => '/var/log/honeyphp/honeyphp_detailed.log'
];

@mkdir(dirname($CONFIG['log_file']), 0777, true);

function log_attack($type, $details) {
    global $CONFIG;
    $timestamp = date('Y-m-d H:i:s');
    $client_ip = get_client_ip();
    $log_entry = "[{$timestamp}] [{$type}] IP: {$client_ip} | Details: " . json_encode($details) . "\n";
    file_put_contents($CONFIG['log_file'], $log_entry, FILE_APPEND);
}

function get_client_ip() {
    if (!empty($_SERVER['HTTP_CLIENT_IP'])) {
        $ip = $_SERVER['HTTP_CLIENT_IP'];
    } elseif (!empty($_SERVER['HTTP_X_FORWARDED_FOR'])) {
        $ip = explode(',', $_SERVER['HTTP_X_FORWARDED_FOR'])[0];
    } else {
        $ip = $_SERVER['REMOTE_ADDR'];
    }
    return $ip;
}

header('Server: Apache');
header("X-elastic-product: Elasticsearch");

$request_method = $_SERVER['REQUEST_METHOD'];
$request_uri = $_SERVER['REQUEST_URI'];
$request_path = parse_url($request_uri, PHP_URL_PATH);
$request_query = parse_url($request_uri, PHP_URL_QUERY);
$request_body = file_get_contents('php://input');

if ($request_method === 'GET') {
    handle_get_request($request_path, $request_query, $request_uri);
} elseif ($request_method === 'POST') {
    handle_post_request($request_path, $request_body);
} else {
    http_response_code(405);
    echo json_encode(['error' => 'Method not allowed']);
}
```

```
function handle_get_request($path, $query, $request_uri) {
    global $CONFIG;

    if ($path === '/' || $path === '' || $path === '/index.php') {
        header('Content-Type: application/json');
        http_response_code(200);
        log_attack('INFO_REQUEST', ['type' => 'root', 'path' => $path]);
        echo json_encode([
            'name' => $CONFIG['node_name'],
            'cluster_name' => $CONFIG['cluster_name'],
            'cluster_uuid' => 'xQv_8aSuQ6OU-bAW4y5Vxg',
            'version' => [
                'number' => $CONFIG['version'],
                'build_flavor' => 'default',
                'build_type' => 'docker',
                'build_hash' => '7ec3a03',
                'build_date' => '2021-04-19T20:42:05.669758Z',
                'build_snapshot' => false,
                'lucene_version' => '9.5.0',
                'minimum_wire_compatibility_version' => '7.17.0',
                'minimum_index_compatibility_version' => '7.0.0'
            ],
            'tagline' => 'You Know, for Search'
        ]);
        return;
    }

    if (preg_match('#^/_cluster/health#', $path)) {
        header('Content-Type: application/json');
        http_response_code(200);
        log_attack('CLUSTER_HEALTH', ['path' => $path]);
        echo json_encode([
            'cluster_name' => $CONFIG['cluster_name'],
            'status' => 'green',
            'timed_out' => false,
            'number_of_nodes' => 3,
            'number_of_data_nodes' => 3,
            'active_primary_shards' => 45,
            'active_shards' => 135,
            'relocating_shards' => 0,
            'initializing_shards' => 0,
            'unassigned_shards' => 0,
            'delayed_unassigned_shards' => 0,
            'number_of_pending_tasks' => 0,
            'number_of_in_flight_fetch' => 0,
            'task_max_waiting_in_queue_millis' => 0,
            'active_shards_percent_as_number' => 100.0
        ]);
        return;
    }

    if (preg_match('#^/_nodes#', $path)) {
        header('Content-Type: application/json');
        http_response_code(200);
        log_attack('NODES_INFO', ['path' => $path]);
        echo json_encode([
            '_nodes' => ['total' => 3, 'successful' => 3, 'failed' => 0],
            'cluster_name' => $CONFIG['cluster_name'],
            'nodes' => [
                'dXpvUEsyUXF6TjRSODEyN2Z2MFg=' => [
```

```
        echo json_encode([
            '_nodes' => ['total' => 3, 'successful' => 3, 'failed' => 0],
            'cluster_name' => $CONFIG['cluster_name'],
            'nodes' => [
                'dXpvUEsyUXF6TjRSODEyN2Z2MFg=' => [
                    'name' => 'node-1',
                    'transport_address' => '172.17.0.2:9300',
                    'host' => '172.17.0.2',
                    'ip' => '172.17.0.2',
                    'version' => $CONFIG['version'],
                    'build_hash' => '7ec3a03',
                    'http_address' => '172.17.0.2:9200'
                ],
                'node2_id' => [
                    'name' => 'node-2',
                    'transport_address' => '172.17.0.3:9300',
                    'host' => '172.17.0.3',
                    'ip' => '172.17.0.3',
                    'version' => $CONFIG['version'],
                    'build_hash' => '7ec3a03',
                    'http_address' => '172.17.0.3:9200'
                ],
                'node3_id' => [
                    'name' => 'node-3',
                    'transport_address' => '172.17.0.4:9300',
                    'host' => '172.17.0.4',
                    'ip' => '172.17.0.4',
                    'version' => $CONFIG['version'],
                    'build_hash' => '7ec3a03',
                    'http_address' => '172.17.0.4:9200'
                ]
            ]
        ]);
        return;
    }

    if (preg_match('#/_search|/user/_search#', $path)) {
        header('Content-Type: application/json');
        http_response_code(200);

        if (stripos($request_uri, 'script') !== false || stripos($request_uri, 'eval') !== false || stripos($request_uri, 'painless') !== false) {
            log_attack('RCE_ATTEMPT', ['path' => $path, 'query' => $query, 'type' => 'script_injection']);
        } else {
            log_attack('SEARCH_QUERY', ['path' => $path, 'query' => $query, 'type' => 'user_search']);
        }
```

```
    echo json_encode([
        'took' => 45,
        'timed_out' => false,
        '_shards' => ['total' => 5, 'successful' => 5, 'skipped' => 0, 'failed' => 0],
        'hits' => [
            'total' => ['value' => 1234, 'relation' => 'eq'],
            'max_score' => 1.0,
            'hits' => [
                [
                    '_index' => 'user',
                    '_type' => '_doc',
                    '_id' => '1',
                    '_score' => 1.0,
                    '_source' => [
                        'id' => 1,
                        'username' => 'admin',
                        'email' => 'admin@example.com',
                        'surname' => 'Administrator',
                        'created_at' => '2021-01-01T00:00:00Z'
                    ]
                ],
                [
                    '_index' => 'user',
                    '_type' => '_doc',
                    '_id' => '2',
                    '_score' => 0.9,
                    '_source' => [
                        'id' => 2,
                        'username' => 'user123',
                        'email' => 'user@example.com',
                        'surname' => 'TestUser',
                        'created_at' => '2021-01-02T00:00:00Z'
                    ]
                ]
            ]
        ]
    ]);
    return;
}

if (preg_match('#^/([a-z0-9_-]+)/?$#i', $path, $matches)) {
    header('Content-Type: application/json');
    http_response_code(200);
    $index = $matches[1];
    log_attack('INDEX_INFO', ['index' => $index]);
```

```
        echo json_encode([
            $index => [
                'aliases' => [],
                'mappings' => [
                    'properties' => [
                        'id' => ['type' => 'integer'],
                        'username' => ['type' => 'keyword'],
                        'email' => ['type' => 'keyword'],
                        'surname' => ['type' => 'text'],
                        'created_at' => ['type' => 'date']
                    ]
                ],
                'settings' => [
                    'index' => [
                        'creation_date' => '1609459200000',
                        'number_of_shards' => '5',
                        'number_of_replicas' => '1',
                        'uuid' => 'a1b2c3d4e5f6g7h8'
                    ]
                ]
            ]
        ]);
        return;
    }

    if (preg_match('#^/([a-z0-9_-]+)/([a-z0-9_-]+)/?$#i', $path, $matches)) {
        header('Content-Type: application/json');
        http_response_code(404);
        $index = $matches[1];
        $field = $matches[2];
        log_attack('CUSTOM_QUERY', ['index' => $index, 'field' => $field, 'path' => $path]);

        echo json_encode([
            'error' => [
                'root_cause' => [
                    [
                        'type' => 'index_not_found_exception',
                        'reason' => "no such index [{$index}]",
                        'resource.id' => $index,
                        'resource.type' => 'index_or_alias',
                        'index' => $index
                    ]
                ],
                'type' => 'index_not_found_exception',
                'reason' => "no such index [{$index}]",
                'index' => $index
            ],
            'status' => 404
        ]);
        return;
    }

    header('Content-Type: application/json');
    http_response_code(404);
    echo json_encode(['error' => 'Not Found']);
}
```

```
function handle_post_request($path, $body) {
    global $CONFIG;

    log_attack('POST_REQUEST', [
        'path' => $path,
        'body_length' => strlen($body),
        'body_preview' => substr($body, 0, 200)
    ]);

    if (stripos($body, 'script') !== false || stripos($body, 'eval') !== false || stripos($body, 'painless') !== fals
                                                                                                                    e) {
        log_attack('RCE_ATTEMPT_POST', ['path' => $path, 'body_snippet' => substr($body, 0, 500)]);
    }

    if (preg_match('#/_search|/([a-z0-9_-]+)/_search#', $path)) {
        header('Content-Type: application/json');
        http_response_code(200);

        echo json_encode([
            'took' => 78,
            'timed_out' => false,
            '_shards' => ['total' => 5, 'successful' => 5],
            'hits' => [
                'total' => ['value' => 2456, 'relation' => 'eq'],
                'max_score' => null,
                'hits' => []
            ]
        ]);
        return;
    }

    header('Content-Type: application/json');
    http_response_code(200);
    echo json_encode(['acknowledged' => true]);
}
?>
```

I prepared the environment for the PHP honeypot by creating the /var/log/honeyphp directory and initializing the necessary log files with appropriate permissions to ensure data could be recorded correctly. To ensure the honeypot remains active and persistent as required by the assignment, I created a systemd service file that manages the PHP server on port 9500 and includes an automatic restart policy.

```
root@localhost:/home/honeyphp# mkdir -p /var/log/honeyphp
root@localhost:/home/honeyphp# chmod 777 /var/log/honeyphp
root@localhost:/home/honeyphp# touch /var/log/honeyphp/honeyphp.log
root@localhost:/home/honeyphp# touch /var/log/honeyphp/honeyphp_detailed.log
root@localhost:/home/honeyphp# chmod 666 /var/log/honeyphp/*
root@localhost:/home/honeyphp# tee /etc/systemd/system/honeyphp.service > /dev/null <<'SERVICE_EOF'
[Unit]
Description=Honeyphp - Enhanced Elasticsearch Honeypot
After=network.target
StartLimitIntervalSec=0

[Service]
Type=simple
User=root
WorkingDirectory=/home/honeyphp
ExecStart=/usr/bin/php -S 0.0.0.0:9500 -t /home/honeyphp
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
SERVICE_EOF
```

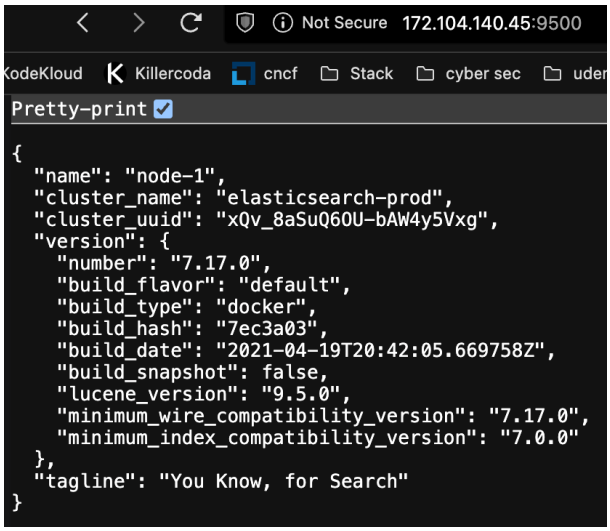
To finalize the deployment, I reloaded the system manager configuration and enabled the honeyphp.service to ensure it starts automatically upon system boot. I then started the service and verified its operational status using the systemctl status command. The output confirms that the "Enhanced Elasticsearch Honeypot" is active (running) on the designated port 9500, with the PHP development server successfully handling the process in the background. This implementation ensures the honeypot remains active and persistent for the duration of the course as required by the assignment guidelines.

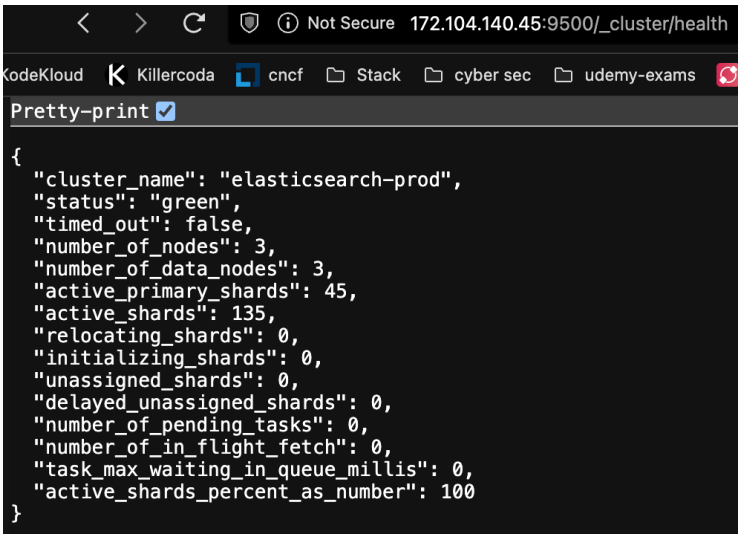
```
root@localhost:/home/honeyphp# systemctl daemon-reload && systemctl enable honeyphp && systemctl start honeyphp
Created symlink /etc/systemd/system/multi-user.target.wants/honeyphp.service → /etc/systemd/system/honeyphp.service.
root@localhost:/home/honeyphp# systemctl status honeyphp
● honeyphp.service - Honeyphp - Enhanced Elasticsearch Honeypot
     Loaded: loaded (/etc/systemd/system/honeyphp.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-04-27 11:40:58 UTC; 9s ago
   Main PID: 1065 (php)
      Tasks: 1 (limit: 2263)
     Memory: 6.6M (peak: 6.7M)
        CPU: 31ms
     CGroup: /system.slice/honeyphp.service
             └─1065 /usr/bin/php -S 0.0.0.0:9500 -t /home/honeyphp

Apr 27 11:40:58 localhost systemd[1]: Started honeyphp.service - Honeyphp - Enhanced Elasticsearch Honeypot.
Apr 27 11:40:58 localhost php[1065]: [Mon Apr 27 11:40:58 2026] PHP 8.3.6 Development Server (http://0.0.0.0:9500) started
```

I verified the functionality of the PHP-based honeypot on port 9500 by testing core endpoints through a web browser. The root directory successfully displayed the "elasticsearch-prod" cluster name and

version 7.17.0, matching the expected output of a modern instance. The /_cluster/health endpoint correctly returned a "green" status across three nodes, effectively simulating a healthy production environment. Finally, I confirmed the search functionality via /user/_search, which served realistic mock data for administrative and user accounts. These tests demonstrate that the custom implementation accurately replicates the decoy features required for the task while maintaining a high level of technical credibility.





I verified the logging functionality by examining the honeyphp.log file using the cat command. The log successfully captured all testing interactions, including the initial root request, the /user/_search query with the size=1000 parameter, and multiple /_cluster/health checks. Each entry provides structured data including precise timestamps, categorized request types, and the source IP address 24.133.168.253, confirming that the logging system is operating as intended.

```
root@localhost:~# cat /var/log/honeyphp/honeyphp.log
[2026-04-27 11:41:15] [INFO_REQUEST] IP: 24.133.168.253 | Details: {"type":"root","path":"\/"}
[2026-04-27 11:42:38] [SEARCH_QUERY] IP: 24.133.168.253 | Details: {"path":"\/user\/_search","query":"size=1000","type":"user_search"}
[2026-04-27 11:43:19] [CLUSTER_HEALTH] IP: 24.133.168.253 | Details: {"path":"\/_cluster\/health"}
[2026-04-27 11:50:54] [CLUSTER_HEALTH] IP: 24.133.168.253 | Details: {"path":"\/_cluster\/health"}
[2026-04-27 12:06:16] [CLUSTER_HEALTH] IP: 24.133.168.253 | Details: {"path":"\/_cluster\/health"}
```

To ensure continuous operation and persistent data collection after closing the terminal session, I deployed the honeypots in detached background modes. Using the screen -d -m command, I created a persistent session for ElasticPot that automatically enters its directory and activates the virtual environment to run the Python script. I applied a similar detached session for Elastichoney to keep the binary running with its specific configuration file. You can access these active services at any time by reattaching to the "elasticpot" or "elastichoney" sessions, ensuring they remain online to capture attacker interactions until the end of the course.

```
root@localhost:/home# screen -S elasticpot -d -m bash -c "cd /home/elasticpot && source venv/bin/activate && python3 elasticpot.py"
root@localhost:/home# screen -S elastichoney -d -m bash -c "cd /home && ./elastichoney/elastichoney -config ./elastichoney/config.json"
```

To finalize the laboratory requirements, I exported my full command history to a text file. Using the command history > history.txt, I created a comprehensive record of all terminal actions performed during the installation, configuration, and testing of the honeypots. I have included this history.txt file in the main assignment folder alongside the report to provide a complete audit trail of the session.

```
root@localhost:/home# history > history.txt
root@localhost:/home# ls
elastichoney  elasticpot  history.txt  honeyphp
```